

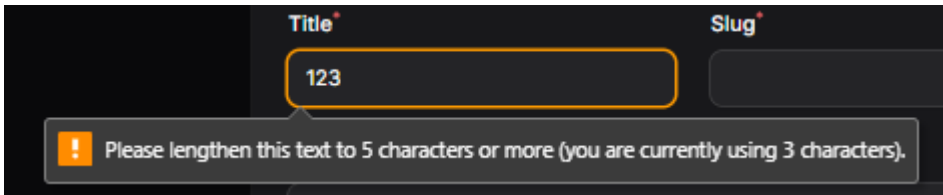
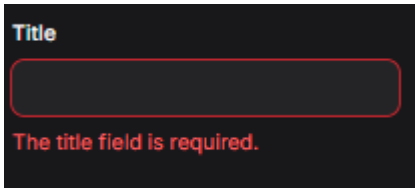
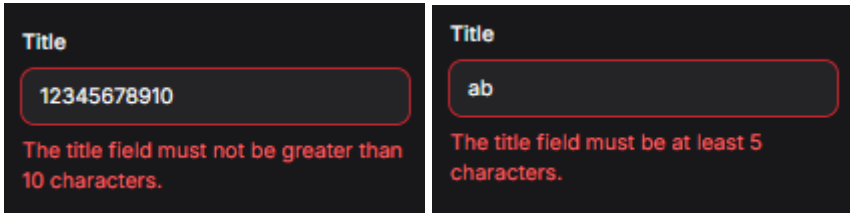
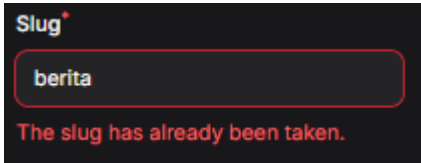


Mata Kuliah : PWL
Program Studi : D4 – Teknik Informatika
Semester : 4

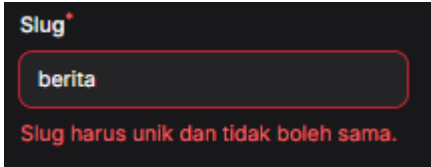
Kelas : TI - 2D
NIM : 244107020207
Nama : Adam Bahy Maulana
Jobsheet Ke- : 6

Laporan Jobsheet

Praktikum Ke-1

Langkah	Jawaban/Deskripsi
1	
2	
3	
D	



E	
---	---

Latihan

Langkah	Jawaban/Deskripsi
1	<pre>])TextInput::make('title') ->rules(['required', 'min:5', 'max:10']) ->validationMessages(['required' => 'Judul harus diisi.', 'min' => 'Judul minimal 5 karakter.', 'max' => 'Judul maksimal 10 karakter.',]), TextInput::make('slug') ->required() ->minLength(3) ->unique(ignoreRecord: true) ->validationMessages(['unique' => 'Slug harus unik dan tidak boleh sama.', 'required' => 'Slug harus diisi.', 'minLength' => 'Slug minimal 3 karakter.',])</pre>



2

The screenshot displays a content management system interface with several validation errors highlighted in red. The main section, titled 'Konten Utama', contains fields for 'Title' (with value '123' and error 'The title field must be at least 5 characters.'), 'Slug' (with value 'berita' and error 'Slug harus unik dan tidak boleh sama.'), 'Kategori' (with a dropdown menu and error 'The kategori field is required.'), and 'Body' (with a rich text editor and error 'The body field is required.'). To the right, the 'Media' section has an 'Image' field with error 'The image field is required.' and the 'Pengaturan & Meta' section has a 'Warna Tema' field with error 'Atur visibilitas dan label.' and a 'Tags' field with error 'Tambah tag...'. Below the main form, there is a separate 'Title' field with error 'Judul harus diisi.'

Diskusi

Langkah	Jawaban/Deskripsi
1	Integritas Data: Memastikan bahwa data yang masuk ke database sesuai dengan format yang diharapkan (misalnya: slug harus unik, judul minimal 5 karakter). Keamanan: Mencegah input berbahaya atau tidak sengaja yang bisa merusak sistem atau menyebabkan error database, seperti QueryException yang terjadi jika kolom tidak ditemukan atau data terlalu panjang. Pengalaman Pengguna (UX): Memberikan umpan balik langsung kepada admin jika ada kesalahan pengisian sebelum data tersebut diproses lebih jauh.



2	Client-side Validation: Terjadi di browser pengguna (seperti atribut required pada HTML atau pengecekan JavaScript). Ini memberikan respon instan namun mudah dimanipulasi oleh pengguna ahli, sehingga tidak cukup untuk keamanan. Server-side Validation: Terjadi di sisi server (Laravel/Filament). Ini adalah validasi mutlak yang memeriksa data sebelum masuk ke database. Meskipun lebih aman, validasi ini memerlukan pengiriman data ke server terlebih dahulu. Filament menggabungkan keduanya secara efisien menggunakan Livewire.
3	Cara Kerjanya: Saat membuat data baru, sistem memeriksa apakah slug sudah ada di database. Saat Edit: Jika kita tidak menggunakan ignoreRecord: true, sistem akan mendeteksi slug milik data itu sendiri sebagai "sudah ada", sehingga validasi gagal. Dengan ignoreRecord: true, Filament secara otomatis memerintahkan Laravel untuk mengecualikan ID data yang sedang diedit dari pengecekan keunikan tersebut.
4	Logika Kompleks: Jika Anda menggunakan objek validasi kustom atau logika bersyarat yang tidak bisa direpresentasikan dengan string sederhana. Kejelasan: Array lebih mudah dibaca dan dikelola jika daftar validasi sangat panjang. Karakter Khusus: Jika nilai validasi mengandung karakter pipa (), maka format string akan pecah, sehingga format array menjadi wajib digunakan.